

Simple Shop Architecture

Simple Shop — Technical Architecture Documentation

Project: next14-ecommerce-starter
Stack: Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, Supabase (Auth + Postgres), Stripe Checkout, OpenAI (gpt-4o-mini)
Purpose: Architecture review reference — exhaustive inventory of structure, APIs, data, flows, env, debt, and Azure ops.

Table of Contents

- 1. [System Overview](#)
 - 2. [Full Project Structure](#)
 - 3. [Pages & Routes \(UI\)](#)
 - 4. [API Routes](#)
 - 5. [Database Schema](#)
 - 6. [Authentication Flow](#)
 - 7. [Cart State Management](#)
 - 8. [Payment Flow](#)
 - 9. [AI Recommendations Flow](#)
 - 10. [Stock Management](#)
 - 11. [Environment Variables](#)
 - 12. [Known Limitations & Technical Debt](#)
 - 13. [Azure Deployment Configuration](#)
-

1. System Overview

Concern	Implementation
UI	Server + client components, Tailwind
Catalog	Supabase products, fetched via API or server <code>getProducts()</code>
Auth	Supabase email/password, cookie sessions via <code>@supabase/ssr</code>
Cart	React Context + <code>localStorage</code>

Concern	Implementation
Checkout	Stripe Checkout Session (server-created redirect URL)
Order persistence	Client calls <code>POST /api/stripe-success</code> after redirect (no webhooks)
Stock decrement	Service-role admin client or RPC fallback
Recommendations	<code>POST /api/recommendations</code> → OpenAI Chat Completions

High-level data flow: Browser ↔ Next.js App ↔ Supabase / Stripe / OpenAI. Cart persists in `localStorage`. Stripe redirects back to `/success`, which triggers order save and stock decrement.

2. Full Project Structure

Every application file (excluding `.git/`, `node_modules/`, `.next/`).

```

next14-ecommerce-starter/
├── .env.example           # Template for required environment
variables
├── .gitignore           # Ignores node_modules, .next,
.env.local
├── README.md            # Setup, Supabase, Stripe, run
instructions
├── middleware.ts         # Next.js middleware entry; delegates to
Supabase session refresh
├── next.config.mjs       # Next image remotePatterns for
images.unsplash.com
├── next-env.d.ts         # Auto-generated Next.js TypeScript
references
├── package.json          # Dependencies and npm scripts
(dev/build/start/lint)
├── package-lock.json     # Locked dependency tree
├── postcss.config.js     # PostCSS with Tailwind
├── tailwind.config.ts    # Tailwind content paths (app,
components, lib)
├── tsconfig.json         # Strict TS, path alias @/* → project
root
|
├── app/                 # Next.js App Router
|   ├── globals.css      # Tailwind directives, light color-
scheme, base link styles
|   ├── layout.tsx       # Root layout: Inter font, AuthProvider,
CartProvider, Navbar
|   ├── page.tsx         # Home: hero + ProductBrowser
|   └── api/             # Route Handlers (REST-style JSON APIs)
|       ├── checkout/route.ts
|       └── orders/route.ts

```

```

|   |   |— products/route.ts
|   |   |— products/[id]/route.ts
|   |   |— recommendations/route.ts
|   |   |— stripe-success/route.ts
|
|   |— cart/page.tsx          # Cart page: line items + CartSummary
|   |— cancel/page.tsx       # Static “payment cancelled” page
|   |— success/page.tsx      # Post-payment: clear cart, POST stripe-
success
|   |— order-summary/page.tsx # Checkout form; requires login; starts
Stripe
|   |— orders/page.tsx        # Order history (client fetch GET
/api/orders)
|   |— login/page.tsx         # Login wrapper around AuthForm
|   |— signup/page.tsx        # Signup wrapper around AuthForm
|   |— products/[id]/
|       |— page.tsx           # Server component: getProductById →
ProductDetail
|       |— loading.tsx        # Route loading UI
(ProductDetailSkeleton)
|       |— not-found.tsx      # 404 UI for unknown product IDs
|
|— components/
|   |— add-to-cart-button.tsx # Stock-aware add button; disabled
states
|   |— auth-form.tsx          # Login/signup form (Supabase client
auth)
|   |— auth-provider.tsx      # Auth context: user, isReady,
signOut
|   |— cart-context.tsx       # Cart state, localStorage, totals,
stock enforcement
|   |— cart-item-row.tsx      # Single cart line: qty input,
remove
|   |— cart-summary.tsx       # Sidebar totals + link to order
summary
|   |— navbar.tsx             # Nav links, cart count, auth UI,
logout
|   |— product-browser.tsx    # Home catalog: fetch, search,
category filter
|   |— product-card.tsx       # Product tile + StockStatus +
AddToCartButton
|   |— product-detail.tsx     # PDP: image, price, StockStatus,
recommendations
|   |— product-recommendations.tsx # Client: fetch products + POST
recommendations
|   |— skeleton.tsx           # Loading skeletons
|   |— stock-status.tsx       # Out of Stock / Only X left / In
Stock badge
|
|— lib/

```

```

|   |— money.ts                # formatMoney() – USD
Intl.NumberFormat
|   |— order-utils.ts         # Totals, parse Stripe metadata
items
|   |— orders-server.ts       # markOrderStockDecrement()
|   |— products.ts            # getProducts(), getProductById()
|   |— stock.ts               # Stock display, cart rules,
decrementStockForOrder()
|   |— stripe.ts              # getStripe(), getAppBaseUrl()
|   |— stripe-client.ts       # loadStripe() singleton (unused in
checkout flow)
|   |— supabase.ts            # Deprecated getSupabase() wrapper
|   |— types.ts               # Product, CartItem, Order types
|   |— supabase/
|       |— admin.ts           # Service-role Supabase client
(server-only)
|       |— client.ts          # Browser Supabase client
|       |— middleware.ts      # updateSession(): refresh cookies,
protect /orders
|       |— server.ts          # Server Supabase client (cookies)
|— supabase/                  # SQL migrations (run manually in
Supabase SQL editor)
    |— schema.sql
    |— add-order-stock-email.sql
    |— add-stock-quantity.sql
    |— auth-orders-policy.sql
    |— fix-canvas-tote-image.sql

```

3. Pages & Routes (UI)

Path	Type	Auth	Description
/	Client-heavy	Public	Product catalog with search/filter
/products/[id]	Server + client	Public	Product detail; SSR product fetch
/cart	Client	Public	Cart management
/order-summary	Client	Login required (client redirect)	Customer form + Stripe redirect
/success	Client	Session needed for order save	Payment confirmation
/cancel	Static	Public	Cancel messaging
/login	Client	Public	Email/password login
/signup	Client	Public	Email/password signup

Path	Type	Auth	Description
/orders	Client	Middleware + session	Order history

4. API Routes

GET /api/products

Auth	None
Returns	200 { products: Product[] } or 500 { message }
External	Supabase products

GET /api/products/[id]

Params	id — product UUID
Returns	200 { product }, 404, 500
External	Supabase

GET /api/orders

Auth	Supabase session (401 if missing)
Returns	200 { orders: Order[] }
External	Supabase, filtered by email = user.email

POST /api/orders

Auth	None
Body	{ items: CartItem[], customer: { name, email, address } }
Returns	200 { orderId, order }
Note	Legacy path; Stripe flow uses stripe-success

POST /api/checkout

Body	{ items: MinimalCartItem[], customer }
Returns	200 { url } — Stripe Checkout URL
External	Stripe checkout.sessions.create

Stripe session: mode: payment, USD, line items + Shipping (\$6) + Tax (8%). Metadata: customer_name, customer_email, shipping_address, items_json. Success URL: /success?session_id={CHECKOUT_SESSION_ID}. Cancel URL: /cart.

POST /api/stripe-success

Auth	Required
Body	{ session_id: string }
Returns	200 { orderId }
External	Stripe retrieve session; Supabase insert/update; stock decrement

Idempotent via stripe_session_id and stock_decremented flag.

POST /api/recommendations

Body	{ currentProduct, allProducts } (name, category, description each)
Returns	200 { recommendations: [{ name, reason }] } (max 3)
External	OpenAI gpt-4o-mini chat completions

5. Database Schema

Table: products

Column	Type	Notes
id	uuid	PK, default gen_random_uuid()
name	text	NOT NULL
description	text	NOT NULL
price	numeric	NOT NULL
category	text	NOT NULL
image_url	text	NOT NULL
stock_quantity	integer	NOT NULL, >= 0

RLS: Public SELECT only.

Table: orders

Column	Type	Notes
id	uuid	PK
customer_name	text	NOT NULL

Column	Type	Notes
email	text	NOT NULL
shipping_address	text	NOT NULL
total	numeric	NOT NULL
items	jsonb	NOT NULL
created_at	timestampz	default now()
stripe_session_id	text	UNIQUE
stock_decremented	boolean	default false

RLS: Public INSERT; SELECT/UPDATE own rows by JWT email.

Function: `decrement_product_stock(uuid, integer)`

SECURITY DEFINER; reduces stock; granted to authenticated.

6. Authentication Flow

Signup: `supabase.auth.signUp` → session or email confirmation message.

Login: `signInWithPassword` → `redirect` (default `/orders`) + `router.refresh()`.

Logout: `signOut` → clear user → **cart cleared** on user→null transition.

Session: HTTP cookies via `@supabase/ssr`. Middleware refreshes session; protects `/orders`.

Clients: `lib/supabase/client.ts` (browser), `server.ts` (RSC/API), `middleware.ts`.

`/order-summary` uses client-side redirect only (not middleware).

7. Cart State Management

Key: `simple-shop-cart` in `localStorage`.

Item shape: Full Product fields + quantity (includes snapshot `stockQuantity`).

Event	Behavior
Mount	Load from <code>localStorage</code>
items change	Persist JSON (remove key if empty)
Logout	<code>clearCart()</code>
Login	Cart preserved
<code>/success</code>	<code>clearCart()</code> immediately

Totals: subtotal; shipping \$6 if subtotal > 0; tax 8%; total = sum.

sessionStorage: order-saved-{sessionId} prevents duplicate stripe-success calls.

8. Payment Flow

1. User adds items (localStorage-backed cart).
2. /order-summary — must be logged in.
3. POST /api/checkout → Stripe session URL.
4. Browser redirects to Stripe Checkout.
5. On pay → /success?session_id=...
6. Cart cleared immediately.
7. POST /api/stripe-success — verify paid, insert order, decrement stock.
8. /orders shows history via GET /api/orders.

No Stripe webhooks — client must reach success page.

9. AI Recommendations Flow

1. PDP loads ProductRecommendations.
2. GET /api/products → filter out current product.
3. POST /api/recommendations with current + all others.

System prompt:

You are a smart ecommerce recommendation engine.
Recommend 2–3 complementary products from the provided allProducts list that pair well with the currentProduct.
Only recommend products that exist in allProducts (use their exact name values).
Return JSON with this exact structure: { "recommendations": [{ "name": string, "reason": string }] }

OpenAI: gpt-4o-mini, temperature 0.7, response_format: json_object.

Response: Validated names only; max 3; errors fail silently on UI.

10. Stock Management

stockQuantity	Display
0	Out of Stock
1–10	Only X left
11+	In Stock badge

Decrement: Admin service role (preferred) or RPC fallback after order save.

11. Environment Variables

Variable	Required	Exposure	Purpose
NEXT_PUBLIC_SUPABASE_URL	Yes	Public	Supabase URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Yes	Public	Anon key
SUPABASE_SERVICE_ROLE_KEY	Strongly recommended	Server only	Stock update
STRIPE_SECRET_KEY	Yes	Server	Checkout
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Optional	Public	Stripe client (unused)
NEXT_PUBLIC_APP_URL	Yes (prod)	Public	Redirect URLs
OPENAI_API_KEY	Yes (recomm)	Server	OpenAI
OPEN_AI_API_KEY	Optional	Server	Alias
VERCEL_URL	Auto	Server	Fallback URL

12. Known Limitations & Technical Debt

- No Stripe webhooks — order save depends on client /success call.
 - Order email stored as logged-in user email, not checkout form email.
 - Public POST /api/orders and public orders INSERT policy.
 - Client-trusted prices at checkout; no server cart re-validation.
 - Stale stockQuantity in localStorage cart items.
 - lib/stripe-client.ts unused; lib/supabase.ts deprecated.
 - Hardcoded \$6 shipping and 8% tax in multiple places.
 - No rate limiting on APIs.
 - Azure/CI not in repo.
-

13. Azure Deployment Configuration

Not in repository — configured via Azure CLI.

Setting	Value
App Service	hemang-ecomm-app
Resource group	ecomm-app-rg
URL	https://hemang-ecomm-app.azurewebsites.net

Configure settings:

```
az webapp config appsettings set --name hemang-ecomm-app --resource-group ecomm-app-rg --settings KEY="value"
az webapp restart --name hemang-ecomm-app --resource-group ecomm-app-rg
```

Expected app settings: Supabase URL/keys, OpenAI, Stripe, NEXT_PUBLIC_APP_URL, SUPABASE_SERVICE_ROLE_KEY.

Redeploy via Azure Portal Deployment Center, Git push, or ZIP deploy (not codified in repo).

Appendix: npm Scripts

Script	Command
dev	next dev
build	next build
start	next start
lint	next lint

Appendix: Dependencies

next ^14.2, react ^18.3, @supabase/supabase-js, @supabase/ssr, stripe, @stripe/stripe-js, tailwindcss.